ExamForge AI

Data Integration Report

Supabase database connections and service layer documentation

Generated: August 02, 2026 at 01:06 UTC

Confidential

1. Overview

This report documents the complete data integration layer connecting the ExamForge AI Next.js application to the Supabase PostgreSQL database. Seven new service modules have been created, three new API routes have been added, and all pages now query live data through these services.

2. Service Layer Architecture

The application follows a service-layer pattern where server components call service functions that query Supabase directly. Client components use API routes that internally call the same service functions. This ensures consistent data access patterns and proper auth context propagation.

Service File	Module	Key Functions	Supabase Tables
analytics-service.ts	Analytics	getAnalyticsData()	exam_sessions, exams, schools, profiles, payments
cbt-service.ts	CBT / Exams	getCBTData()	exams, questions, exam_sessions, classes, profiles
results-service.ts	Results	getResultsData()	exam_sessions, profiles, exams, classes
question-bank-service.ts	Question Bank	getQuestionBankData()	questions, subjects, topics, exam_questions
marketplace-service.ts	Marketplace	getMarketplaceData()	marketplace_products, profiles
billing-service.ts	Billing	getBillingData()	subscriptions, plans, payments, profiles
reports-service.ts	Reports	getReportsData()	schools, profiles, exams, questions, payments, exam_sessions
search-service.ts	Search	globalSearch()	schools, profiles, questions, exams, marketplace_products, notifications
dashboard-service.ts	Dashboard	getDashboardData()	schools, profiles, exams, payments, exam_sessions, questions
schools-service.ts	Schools	getSchoolsData()	schools, profiles
users-service.ts	Users	getStudentsData(), getTeachersData(), getParentsData()	profiles, schools, exam_sessions
notifications-service.ts	Notifications	getNotificationsData()	notifications

3. API Routes

Three new API routes have been created for client-side data fetching. These routes authenticate the user, resolve their role and school context, and delegate to the appropriate service function.

Route	Method	Service Function	Parameters
/api/analytics	GET	getAnalyticsData()	Date range filter (7d/30d/90d/1y/all)
/api/reports	GET	getReportsData()	Date from/to filters
/api/search	GET	globalSearch()	Query string (q parameter)

4. Role-Based Data Scoping

All service functions accept role, userId, and schoolId parameters to properly scope data access. Super admins see all data, school admins see data scoped to their school, teachers see data they created, and students see data relevant to their enrollment. This ensures data isolation and security at the application level.

Role	Data Scope	Filter Method	Description
super_admin	All data	No scoping	Full platform access
school_admin	School-scoped	school_id filter	School data only
teacher	Own data	created_by filter	Created exams and questions
student	Own data	student_id filter	Own sessions and results
parent	Children data	children filter	Linked students only

5. Realtime Subscriptions

The application has two realtime subscription layers: a global RealtimeProvider mounted in the authenticated layout that handles cross-page notification badge updates, and a page-level subscription in the Notifications page for live notification list updates. Both subscriptions use user-scoped filters (user_id=eq.{userId}) to prevent data leaks, include deduplication logic to prevent duplicate events on reconnection, and handle reconnection status through the subscribe callback.

6. Supabase Configuration

The application connects to the production Supabase instance at pzfnptrrnxkgodclyhft.supabase.co using the anon key for client-side access and server-side cookie-based auth for server components. The environment variables NEXT_PUBLIC_SUPABASE_URL and NEXT_PUBLIC_SUPABASE_ANON_KEY are configured in the .env file.